Design Patterns

& SOLID Principles

Complete Interview Guide with EnergyGrid Examples

SECTION 1 — SOLID PRINCIPLES

SOLID is a set of 5 object-oriented design principles introduced by Robert C. Martin (Uncle Bob). Following SOLID makes code maintainable, extensible, and testable. Every principle is actively applied in EnergyGrid's Spring Boot architecture.

S

Single Responsibility Principle (SRP)

A class should have only ONE reason to change — it should do only one thing. If a class handles multiple responsibilities, changes to one responsibility may break others.

Violation example:

A class called BillingAndNotificationAndReportService does billing, sends emails, generates PDF reports, and saves audit logs — if email logic changes, you risk breaking billing.

EnergyGrid example (correct):

In EnergyGrid: BillingService ONLY handles bill calculation and status. NotificationService ONLY handles sending and reading notifications. AuditLogService ONLY handles audit logging. EnergyReportService ONLY generates analytics reports. Each service has ONE responsibility — one reason to change.

O

Open/Closed Principle (OCP)

Software entities should be OPEN for extension but CLOSED for modification. You should be able to add new behaviour without editing existing tested code.

Violation example:

A single generateBill() method with if-else blocks for RESIDENTIAL, COMMERCIAL, INDUSTRIAL pricing — adding a new category requires modifying the existing method, risking bugs.

EnergyGrid example (correct):

In EnergyGrid: TariffService uses the SlabRates JSON stored in the Tariff entity. Adding a new consumer category or new pricing tiers requires adding a new Tariff record (extension) without touching BillingService code (closed). In Spring: you can add a new @Service for a new payment method without changing PaymentController.

L

Liskov Substitution Principle (LSP)

Objects of a subclass should be substitutable for objects of their parent class without breaking the program. Subclasses must honour the contract of the parent.

Violation example:

A Square extends Rectangle but overrides setWidth() to also set height — this breaks Rectangle's contract where width and height are independent. Code using Rectangle breaks.

EnergyGrid example (correct):

In EnergyGrid: All custom exceptions (ResourceNotFoundException, BadRequestException) extend RuntimeException. GlobalExceptionHandler handles RuntimeException — adding a new custom exception class requires no changes because it substitutes cleanly. Spring: any JpaRepository implementation (SimpleJpaRepository) substitutes for JpaRepository.

I

Interface Segregation Principle (ISP)

Clients should not be forced to depend on interfaces they do not use. It is better to have many small, focused interfaces than one large general-purpose interface.

Violation example:

A single IUserOperations interface has methods: createUser(), deleteUser(), generateBill(), recordPayment(), reportOutage() — classes implementing it must implement all methods, even those irrelevant to them.

EnergyGrid example (correct):

In EnergyGrid: Each Repository interface is focused — BillRepository has only bill-related methods; OutageEventRepository has only outage methods. No repository implements unrelated methods. Spring Security: UserDetailsService has only loadUserByUsername() — a single-method interface, not a bloated one with roles, permissions, and session management.

D

Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules. Both should depend on abstractions. Abstractions should not depend on details — details should depend on abstractions.

Violation example:

BillingController directly instantiates BillingService: BillingService service = new BillingService(). The controller is tightly coupled — cannot swap implementations or mock in tests.

EnergyGrid example (correct):

In EnergyGrid: BillingController depends on the BillingService abstraction, not a concrete class. Spring injects the implementation via @Autowired. In tests, @Mock creates a mock implementation injected via @InjectMocks — the controller never knows the concrete class. All repositories are interfaces (BillRepository extends JpaRepository) — high-level services depend on the interface, not the concrete SimpleJpaRepository Spring generates at runtime.

SOLID Principles — Quick Reference Table

Principle	Full Name	One Line	EnergyGrid Example
S	**Single Responsibility**	One class = one job	BillingService only bills. AuditLogService only logs. NotificationService only notifies.
O	**Open / Closed**	Open to extend, closed to modify	Add new tariff slabs via data — no code change needed in BillingService.
L	**Liskov Substitution**	Subclass must honour parent contract	ResourceNotFoundException extends RuntimeException — substitutes safely in handler.
I	**Interface Segregation**	Small focused interfaces	BillRepository, OutageEventRepository — each has only its own methods.
D	**Dependency Inversion**	Depend on abstractions	@Autowired BillRepository (interface) — Spring injects concrete impl at runtime.

SECTION 2 — CREATIONAL DESIGN PATTERNS

Creational patterns deal with object creation mechanisms — controlling how objects are instantiated to make systems more flexible and independent of how objects are created, composed, and represented.

1. Singleton Pattern

Singleton

CREATIONAL

Intent: Ensure a class has only ONE instance and provide a global point of access to it.

In EnergyGrid: In EnergyGrid: All Spring @Service and @Repository beans are Singletons by default — Spring creates one instance per ApplicationContext and shares it across all requests. BillingService, PaymentService, AuditLogService — each has exactly one instance. JwtUtil (which holds the secret key) is a Singleton — one instance, one secret. In plain Java: a thread-safe Singleton uses a private static instance and a synchronized getInstance() method.

```
// Spring makes this a singleton automatically @Service public class BillingService { ... } // One instance shared across all HTTP requests
```

Key Interview Points for Singleton:

- Spring beans are Singletons by default (scope=singleton)
- Never store request-specific data in a Singleton field — causes race conditions
- Thread-safe Singleton in plain Java uses double-checked locking or enum
- @Scope('prototype') creates a new instance every time the bean is requested
- JwtUtil, BCryptPasswordEncoder, LoggingAspect — all are Singletons in EnergyGrid

2. Factory Method Pattern

Factory Method

CREATIONAL

Intent: Define an interface for creating objects but let subclasses decide which class to instantiate. The factory method defers instantiation to subclasses.

In EnergyGrid: In EnergyGrid: Spring's ApplicationContext is a factory — you call context.getBean(BillingService.class) and Spring decides which implementation to instantiate. ResponseEntity.ok(), ResponseEntity.notFound(), ResponseEntity.created() are factory methods that create ResponseEntity objects without you calling new directly. Optional.of(), Optional.empty(), Optional.ofNullable() are factory methods for Optional.

```
// Factory method pattern in EnergyGrid ResponseEntity.ok(bill) // creates 200 response ResponseEntity.status(201).body(b) // creates 201 response Optional.of(user) // factory method
```

3. Abstract Factory Pattern

Abstract Factory

CREATIONAL

Intent: Provide an interface for creating families of related or dependent objects without specifying their concrete classes. A factory of factories.

In EnergyGrid: In Spring Security: AuthenticationManagerBuilder is an Abstract Factory that creates different families of authentication mechanisms (in-memory, JDBC, LDAP, JWT) based on configuration. In EnergyGrid: if you had multiple notification channels (email, SMS, push), a NotificationFactory interface with createEmailNotifier() and createSMSNotifier() would return different concrete implementations without the client knowing.

```
// Spring Security uses Abstract Factory http.authenticationProvider(jwtProvider) // Abstract Factory decides which AuthProvider to create
```

4. Builder Pattern

Builder

CREATIONAL

Intent: Separate the construction of a complex object from its representation. Build an object step by step. Avoids telescoping constructors.

In EnergyGrid: In EnergyGrid: @Builder (Lombok) is used on ALL entities — User.builder().name('John').email('j@x.com').role(Role.ADMIN).build(). In JUnit tests: Bill mockBill = Bill.builder().billId(1).accountId(100).status(GENERATED).totalAmount(880.0).build() — readable, no positional constructor confusion. JWT building in JwtUtil uses Jwts.builder() chained calls: .setSubject().claim().setIssuedAt().setExpiration().signWith().compact().

```
// Lombok @Builder in EnergyGrid User user = User.builder() .name("Adithya") .email("a@grid.com") .role(Role.ADMIN) .status(UserStatus.ACTIVE) .build();
```

5. Prototype Pattern

Prototype

CREATIONAL

Intent: Create new objects by copying (cloning) an existing object (the prototype). Avoids expensive object creation when a similar object already exists.

In EnergyGrid: In EnergyGrid: Spring's @Scope('prototype') creates a new bean instance by copying the prototype definition each time. In practice: when copying a Bill entity for a revised bill (credit note), you could clone the original Bill and modify only the amount — implements Cloneable. Lombok's @Builder with toBuilder=true supports this: Bill revisedBill = originalBill.toBuilder().totalAmount(newAmount).build().

```
// Prototype-like copy with Lombok toBuilder Bill revisedBill = originalBill.toBuilder() .totalAmount(newAmount) .status(BillStatus.GENERATED) .build();
```

SECTION 3 — STRUCTURAL DESIGN PATTERNS

Structural patterns deal with how classes and objects are composed to form larger structures. They simplify the structure by identifying relationships.

6. Adapter Pattern

Adapter

STRUCTURAL

Intent: Convert the interface of a class into another interface that clients expect. Lets incompatible interfaces work together.

In EnergyGrid: In EnergyGrid: The DTO (Data Transfer Object) acts as an Adapter between the API layer and the JPA entity layer. BillRequest DTO adapts the JSON input (with String dates) to the Bill entity (with LocalDate). The BillResponse DTO adapts the entity (with Tariff object) to the API output (with just tariffId Integer). Spring's HandlerMethodArgumentResolver adapts HTTP request parameters to Java method parameters.

```
// DTO as Adapter in EnergyGrid BillRequest (API format) --> Adapter (mapper) --> Bill Entity (JPA)
Bill Entity (JPA) --> Adapter (mapper) --> BillResponse (API format)
```

7. Decorator Pattern

Decorator

STRUCTURAL

Intent: Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.

In EnergyGrid: In EnergyGrid: Spring Security's filter chain is a chain of Decorators — each filter wraps the request and adds behaviour (JWT validation, CORS, CSRF) before passing to the next. Spring's @Transactional creates a CGLIB proxy that decorates the original service bean with transaction management behaviour. The JwtFilter decorates every request with authentication. Spring's @Cacheable decorates a method with caching behaviour.

```
// @Transactional is a Decorator @Service public class PaymentService { @Transactional // Spring
wraps this method with transaction decorator public Payment recordPayment(...) { ... } }
```

8. Proxy Pattern

Proxy

STRUCTURAL

Intent: Provide a surrogate or placeholder for another object to control access to it. The proxy has the same interface as the real subject.

In EnergyGrid: In EnergyGrid: Spring AOP (LoggingAspect) creates a CGLIB proxy for every @Service bean. When you call billingService.generateBill(), you are actually calling a Spring proxy object that intercepts the call, runs @Before advice (logs entry), then delegates to the real BillingService. @Transactional also works via a proxy — the proxy manages the transaction. JPA's getReferenceById() returns a Hibernate proxy (lazy-loaded surrogate).

```
// Spring creates a Proxy for every @Service with AOP // What you get from ApplicationContext:
BillingService$EnhancerBySpringCGLIB // proxy // Proxy delegates to real BillingService // and runs
@Before / @AfterReturning advice
```

9. Facade Pattern

Facade

STRUCTURAL

Intent: Provide a simplified interface to a complex subsystem. A facade hides the complexity and provides a simple entry point.

In EnergyGrid: In EnergyGrid: BillingService acts as a Facade for the complex billing subsystem — the controller calls billingService.generateBill() and the facade internally coordinates: fetches Tariff, parses JSON slabs, calculates amount, creates Bill entity, saves, logs audit, sends notification. The controller sees only one method. EnergyReportService is a Facade for querying 5+ repositories and building metrics. Spring's JpaRepository itself is a Facade over complex Hibernate/JPA operations.

```
// BillingService is a Facade billingService.generateBill(accountId, units, tariffId); // Internally: fetch tariff + parse slabs + calculate + save + audit + notify // Controller sees ONE simple method call
```

10. Composite Pattern

Composite

STRUCTURAL

Intent: Compose objects into tree structures to represent part-whole hierarchies. Lets clients treat individual objects and compositions uniformly.

In EnergyGrid: In EnergyGrid: Spring's SecurityFilterChain is a Composite of individual filters treated as a single chain. Spring's ApplicationContext is a Composite of parent and child contexts. Spring MVC's HandlerInterceptorChain is a Composite of interceptors. A zone hierarchy (Region > District > Zone > Consumer) would be a Composite if implemented.

```
// Spring's filter chain is a Composite SecurityFilterChain chain = http .addFilterBefore(jwtFilter, ...) .addFilterAfter(corsFilter, ...) .build(); // composite of all filters
```

SECTION 4 — BEHAVIOURAL DESIGN PATTERNS

Behavioural patterns are concerned with algorithms and the assignment of responsibilities between objects. They describe not just patterns of objects or classes but also the patterns of communication between them.

11. Strategy Pattern

Strategy

BEHAVIOURAL

Intent: Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.

In EnergyGrid: In EnergyGrid: Payment processing uses Strategy — PaymentMethod enum (ONLINE, CASH, CHEQUE, AUTODEBIT) could each have a different PaymentStrategy implementation. In Spring Security: AuthenticationProvider is a Strategy interface — JwtAuthProvider, DaoAuthProvider are concrete strategies. In Spring: sorting strategies (Sort.by(...)), Jackson's serialisation strategy, and Spring Data's query execution strategies are all examples.

```
// Strategy Pattern for Payment interface PaymentStrategy { void process(Payment p); } class
OnlinePaymentStrategy implements PaymentStrategy { ... } class CashPaymentStrategy implements
PaymentStrategy { ... } // PaymentService picks strategy based on PaymentMethod enum
```

12. Observer Pattern

Observer

BEHAVIOURAL

Intent: Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

In EnergyGrid: In EnergyGrid: NotificationService implements Observer pattern — when PaymentService records a payment (subject changes state), it calls NotificationService.sendNotification() to notify the consumer (observer). When OutageEventService reports a new outage, all consumers in that zone are notified. In Spring: ApplicationEventPublisher / @EventListener is the built-in Observer mechanism. React's useState + useEffect is also Observer — component re-renders when state changes.

```
// Spring Events are Observer pattern // Subject: PaymentService publishes event
publisher.publishEvent(new PaymentReceivedEvent(payment)); // Observer: NotificationService listens
@EventListener public void onPayment(PaymentReceivedEvent e) { sendNotification(...); }
```

13. Template Method Pattern

Template Method

BEHAVIOURAL

Intent: Define the skeleton of an algorithm in a base class, deferring some steps to subclasses. Subclasses can redefine certain steps without changing the algorithm structure.

In EnergyGrid: In EnergyGrid: Spring's OncePerRequestFilter uses Template Method — the abstract class defines the template: doFilter() (fixed algorithm skeleton), and defers implementation to doFilterInternal() (abstract method). JwtFilter extends it and overrides only doFilterInternal(). Spring Data JPA's SimpleJpaRepository has template methods. JUnit's @BeforeEach + test methods follow Template Method — setUp is the template step.

```
// Template Method in Spring public abstract class OncePerRequestFilter { // Template: defines the
algorithm structure public void doFilter(req, res, chain) { doFilterInternal(req, res, chain); //
deferred to subclass } protected abstract void doFilterInternal(...); // JwtFilter overrides this }
```

14. Chain of Responsibility Pattern

Chain of Responsibility

BEHAVIOURAL

Intent: Pass a request along a chain of handlers. Each handler decides either to process the request or to pass it to the next handler in the chain.

In EnergyGrid: In EnergyGrid: Spring Security's filter chain is the perfect example — JwtFilter, CorsFilter, ExceptionTranslationFilter, AuthorizationFilter form a chain. Each filter processes the request and calls chain.doFilter() to pass to the next. Spring MVC interceptors (HandlerInterceptorAdapter) form a chain for preHandle/postHandle. GlobalExceptionHandler processes exception types in order — most specific first.

```
// Spring Security is Chain of Responsibility JwtFilter --> CorsFilter -->
ExceptionTranslationFilter --> AuthorizationFilter --> DispatcherServlet // Each calls
filterChain.doFilter() to pass along
```

15. Command Pattern

Command

BEHAVIOURAL

Intent: Encapsulate a request as an object, thereby allowing parameterisation, queuing, logging of requests, and support for undoable operations.

In EnergyGrid: In EnergyGrid: @Transactional with savepoints supports rollback (undo) — a form of Command pattern with undo capability. Spring Batch uses Command pattern — each Step is a command that can be executed, retried, or skipped. AuditLogService records every command (action) as a log entry — audit trail is the log of executed commands. React's form submission: the submit button dispatches a command (action) to the service.

```
// AuditLog records executed commands auditLogService.logAction(userId, "GENERATE_BILL", "Bill",
bill.getBillId()); // Each entry = one command that was executed // Can replay or audit the
sequence of commands
```

16. Iterator Pattern

Iterator

BEHAVIOURAL

Intent: Provide a way to sequentially access elements of an aggregate object without exposing its underlying representation.

In EnergyGrid: In EnergyGrid: Java's for-each loop uses Iterator under the hood — iterating over List<Bill>, List<Notification>. Stream API in Java is a functional Iterator — billList.stream().filter().map().collect(). Spring Data's Page<T> provides iteration over pages of results. In JavaScript: Array.map(), Array.filter(), Array.forEach() are all iterators over the notifications and bills arrays in React components.

```
// Iterator in EnergyGrid — Java streams billList.stream() .filter(b -> b.getStatus() == OVERDUE)
.map(Bill::getBillId) .collect(Collectors.toList()); // React: Array.map() iterates notifications
```

17. State Pattern

State

BEHAVIOURAL

Intent: Allow an object to alter its behaviour when its internal state changes. The object will appear to change its class.

In EnergyGrid: In EnergyGrid: Bill status transitions represent State pattern — a Bill in GENERATED state behaves differently from one in PAID or DISPUTED. Business rules enforced by state: you cannot pay an already PAID bill (BadRequestException). You cannot resolve an already RESOLVED dispute. OutageEvent: ACTIVE → RESTORED — once RESTORED, cannot be restored again. FieldDispatch: DISPATCHED → ONSITE → RESOLVED — forward-only transitions.

```
// State Pattern — bill status controls behaviour if (bill.getStatus() == PAID) throw new
BadRequestException("Already paid"); // GENERATED bill: can pay it // PAID bill: cannot pay again —
different state, different behaviour
```

18. Repository Pattern

Repository

BEHAVIOURAL / ARCHITECTURAL

Intent: Encapsulate data access logic in a repository object. The rest of the application uses the repository to access data without knowing the underlying storage mechanism.

In EnergyGrid: In EnergyGrid: BillRepository, UserRepository, OutageEventRepository — all extend JpaRepository. Services depend on the Repository interface, not the database. If you switch from MySQL to PostgreSQL, only the datasource URL changes — no service code changes. In tests: @Mock BillRepository — the service is tested with a mock repository with no real database. This is DIP + Repository Pattern combined.

```
// Repository Pattern @Repository public interface BillRepository extends JpaRepository<Bill, Long>
{ List<Bill> findByAccountId(Integer accountId); List<Bill> findByStatus(BillStatus status); } //
Services depend on this interface, not MySQL
```

SECTION 5 — DESIGN PATTERNS USED IN SPRING FRAMEWORK

Where Spring Uses Each Pattern

Design Pattern	Where Spring Uses It	EnergyGrid Annotation/Class
Singleton	All @Service, @Repository, @Component beans	@Service BillingService — one instance
Factory Method	ApplicationContext.getBean(), ResponseEntity.ok()	ResponseEntity.ok(bill)
Abstract Factory	FactoryBean, AuthenticationManagerBuilder	SecurityConfig.authManager()
Builder	Jwts.builder(), UriComponentsBuilder	Jwts.builder().setSubject(email).build()
Prototype	@Scope('prototype') beans	@Scope('prototype') — new instance per request
Adapter	HttpMessageConverter, HandlerMethodArgumentResolver	Jackson: JSON to DTO to Entity
Decorator	@Transactional CGLIB proxy, @Cacheable proxy	@Transactional PaymentService
Proxy	Spring AOP CGLIB proxy for @Service beans	LoggingAspect wraps all @Service methods
Facade	JpaRepository, Spring Data template methods	BillingService.generateBill() hides complexity
Observer	ApplicationEventPublisher + @EventListener	@EventListener PaymentReceivedEvent
Template Method	JdbcTemplate, OncePerRequestFilter	JwtFilter extends OncePerRequestFilter
Chain of Responsibility	SecurityFilterChain, HandlerInterceptor chain	JwtFilter → CorsFilter → DispatcherServlet
Command	Spring Batch Step, @Transactional rollback	@Transactional — rollback = undo command
Iterator	Stream API, Page	billList.stream().filter().map()
Strategy	AuthenticationProvider, Sort, SerializationStrategy	Sort.by('timestamp').descending()
Repository	JpaRepository, MongoRepository	BillRepository extends JpaRepository
Dependency Injection	Core IoC container (@Autowired)	@Autowired BillRepository in BillingService
MVC	DispatcherServlet, @Controller, @Service, Repository	Full Spring MVC stack in EnergyGrid

SECTION 6 — SOLID PRINCIPLES — ALL INTERVIEW QUESTIONS & ANSWERS

Single Responsibility Principle

Q1. What is the Single Responsibility Principle? How did you apply it in EnergyGrid?

A: SRP states a class should have only one reason to change — one job. In EnergyGrid, I created separate services for every concern: BillingService only handles bill lifecycle, NotificationService only handles in-app messages, AuditLogService only records audit entries, EnergyReportService only generates analytics. No service does multiple unrelated things. This means when billing logic changes, only BillingService is modified — no risk of accidentally breaking notification or audit functionality.

Q2. Give a real example of an SRP violation and how you would fix it.

A: Violation: A UserService that creates users, sends welcome emails, generates user PDFs, writes audit logs, and manages sessions — 5 different reasons to change. Fix: Split into UserService (user CRUD only), EmailService (email sending), AuditLogService (logging), SessionService (session management). Each class has one reason to change. In EnergyGrid, AuditLogService is intentionally separate from UserService precisely to follow SRP.

Q3. How does SRP relate to cohesion and coupling?

A: SRP promotes HIGH cohesion (all methods in a class are strongly related to its single purpose) and LOW coupling (classes don't need to know about each other's unrelated concerns). A class that does billing AND notifications has low cohesion — its methods are only weakly related. High coupling occurs when changes in one concern force changes in classes that use it for a different concern. SRP helps you achieve the ideal: each class is tightly focused on one thing and loosely connected to others.

Open/Closed Principle

Q4. What is the Open/Closed Principle? How is it achieved in Java?

A: OCP states software entities should be open for extension but closed for modification. In Java, OCP is achieved through: (1) Interfaces and abstract classes — extend by adding new implementations without changing the interface. (2) Polymorphism — add new behaviour via new subclasses. (3) Composition and strategy pattern — inject different behaviours. In EnergyGrid: tariff pricing is open for extension (add new tariff records) without modifying BillingService code.

Q5. How does Spring Framework implement OCP?

A: Spring uses OCP throughout: @Configuration classes let you override auto-configured beans without modifying Spring's source. @Conditional annotations allow extending auto-configuration for new environments. Adding a new @Service for a payment method extends the system without modifying PaymentController. Adding a new @ExceptionHandler in GlobalExceptionHandler extends exception handling without touching existing handlers.

Liskov Substitution Principle

Q6. What is the Liskov Substitution Principle? Give an example violation.

A: LSP states if S is a subtype of T, objects of type T may be replaced with objects of type S without breaking the program. Violation: Square extends Rectangle and overrides setWidth() to also change height — code that does rect.setWidth(5); rect.setHeight(3); assert rect.area() == 15 fails for Square (area becomes 9 because height was overridden). Fix: Square and Rectangle should not have an inheritance relationship — use composition or a common Shape interface without setWidth/setHeight.

Q7. How does EnergyGrid follow LSP in exception handling?

A: ResourceNotFoundException and BadRequestException both extend RuntimeException. GlobalExceptionHandler has @ExceptionHandler methods for each specific type. Any code that catches RuntimeException can handle either — they are substitutable. Spring's @RestControllerAdvice processes them polymorphically — adding a new custom exception that extends RuntimeException works without changing existing

handler code. LSP is maintained: subclasses honour the contract of RuntimeException.

Interface Segregation Principle

Q8. What is the Interface Segregation Principle? How does Spring Data JPA demonstrate it?

A: ISP states clients should not be forced to implement interfaces they do not use. Spring Data JPA demonstrates ISP well: JpaRepository extends PagingAndSortingRepository extends CrudRepository extends Repository. If you only need basic CRUD, extend CrudRepository. If you need pagination, extend PagingAndSortingRepository. If you need all JPA features, extend JpaRepository. You are not forced to implement pagination methods just to get CRUD. In EnergyGrid: BillRepository only declares bill-specific methods — no unrelated methods.

Q9. How do you identify an ISP violation in existing code?

A: Signs: (1) Classes implementing an interface have empty or throw UnsupportedOperationException for several methods — they are forced to implement methods they don't need. (2) An interface has more than 5-7 methods covering very different concerns. (3) When you change one method in the interface, many unrelated implementing classes must be modified. Fix: split the fat interface into smaller role-specific interfaces. Example: split IAllOperations into IReadOperations, IWriteOperations, IDeleteOperations.

Dependency Inversion Principle

Q10. What is the Dependency Inversion Principle? How does Spring's IoC container implement it?

A: DIP states: (1) High-level modules should not depend on low-level modules — both should depend on abstractions. (2) Abstractions should not depend on details. Spring's IoC container implements DIP perfectly: BillingController (high-level) depends on BillingService (abstraction/interface), not on a concrete class. Spring (the container) injects the concrete implementation. The controller never calls new BillingService(). Switching from BillingServiceV1 to BillingServiceV2 requires zero changes in the controller.

Q11. How does DIP make your code more testable? Show an example from your JUnit tests.

A: Because BillingService depends on BillRepository (interface) via @Autowired, in tests I can inject a @Mock BillRepository using @InjectMocks BillingService. The service never knows it has a mock instead of the real repository. when(billRepository.findById(1L)).thenReturn(Optional.of(mockBill)) makes the mock return specific values. If BillingService directly instantiated BillRepository (violating DIP), there would be no way to inject a mock — you would need a real database for every test.

SECTION 7 — DESIGN PATTERNS — ALL INTERVIEW QUESTIONS & ANSWERS

Creational Pattern Questions

Q1. What is the Singleton pattern? How does Spring implement it? What is the risk?

A: Singleton ensures only one instance of a class exists. Spring implements it by default for all @Service, @Repository, @Component beans — one instance per ApplicationContext. Risk: Singleton beans must be STATELESS — never store request-specific data in instance fields. If a Singleton stores state, concurrent requests will share and corrupt it. In EnergyGrid, all services are stateless — they receive parameters, call repositories, and return results without storing intermediate state in fields.

Q2. What is the Factory Method pattern? Give 3 examples from Spring or Java.

A: Factory Method defines an interface for object creation but lets subclasses or static methods decide the class to instantiate. Examples: (1) ResponseEntity.ok(body) — factory creates a ResponseEntity with status 200 without you calling new. (2) Optional.of(value) and Optional.empty() — factories for Optional. (3) Arrays.asList(), List.of(), Map.of() — factory methods that create collections. In Spring: BeanFactory.getBean() and LocalDate.now(), LocalDate.of() in Java.

Q3. What is the Builder pattern? Why is it used in EnergyGrid?

A: Builder separates the construction of a complex object from its representation, building it step by step. In EnergyGrid, Lombok @Builder is on every entity and DTO: User.builder().name('x').email('y').role(ADMIN).build(). Benefits: (1) No telescoping constructor problem with 8+ parameters. (2) Readable — each field is named explicitly. (3) Immutable construction — build() creates the final object. (4) In JUnit tests, mock entities are created with builder very clearly — every test shows exactly which fields are set for the test scenario.

Q4. What is the difference between Builder and Factory Method patterns?

A: Factory Method creates objects in one step — you call a factory method and get a complete object. Suitable when the object creation is simple. Builder constructs complex objects in multiple steps — you set each property, then call build(). Suitable when an object has many optional parameters or requires a specific construction order. Factory focuses on WHAT class to instantiate; Builder focuses on HOW to construct a complex object. Example: ResponseEntity.ok() is Factory (one call). Jwts.builder().setSubject()....compact() is Builder (many steps).

Q5. What is the Prototype pattern? How is @Scope('prototype') different from Singleton?

A: Prototype creates new objects by copying an existing prototype. @Scope('prototype') in Spring creates a new bean instance every time it is requested from the ApplicationContext — unlike Singleton (one shared instance). Use prototype scope when: the bean is stateful and stores request-specific data, you need a fresh instance per request. In EnergyGrid, Singleton is correct for all services (stateless). Prototype would be appropriate for a ReportBuilder bean that accumulates data during a single report generation.

Structural Pattern Questions

Q6. What is the Adapter pattern? How do DTOs act as Adapters in EnergyGrid?

A: Adapter converts the interface of one class into the interface that a client expects. DTOs in EnergyGrid act as Adapters: the REST client speaks in BillRequest JSON format (with String billPeriodStart). The JPA layer speaks in Bill entity format (with LocalDate). The service layer maps between them — BillRequest is adapted to Bill, and Bill is adapted to BillResponse for the response. Jackson's HttpMessageConverter also acts as an Adapter between Java objects and JSON bytes.

Q7. What is the Decorator pattern? How does @Transactional use it?

A: Decorator attaches additional behaviour to an object dynamically without subclassing. @Transactional uses Spring's CGLIB to create a subclass proxy of PaymentService. This proxy (the decorator) wraps every @Transactional method: before calling the real method, it begins a transaction; after successful return, it commits; on exception, it rolls back. The original PaymentService code has no transaction logic — the decorator adds it. Other

Spring decorators: @Cacheable (adds caching), @Async (adds async execution), @Retry (adds retry logic).

Q8. What is the Proxy pattern? How does Spring AOP use it?

A: Proxy provides a surrogate for another object to control access. Spring AOP creates a CGLIB subclass proxy for every @Service bean that has @Aspect advice applied. When you @Autowired BillingService, Spring actually gives you the proxy. Calling generateBill() on the proxy runs: @Before advice (log entry) → real generateBill() → @AfterReturning advice (log exit). The service code never changes — the proxy intercepts. Proxy vs Decorator: Proxy controls access; Decorator adds behaviour. In practice, Spring's AOP proxy does both.

Q9. What is the Facade pattern? How does BillingService demonstrate it?

A: Facade provides a simplified interface to a complex subsystem. BillingService.generateBill() is a Facade — the controller calls one method and the facade internally: fetches the tariff from TariffRepository, parses the JSON slabRates, iterates to find the correct rate, calculates totalAmount with tax, creates a Bill entity, calls billRepository.save(), calls auditLogService.logAction(), and calls notificationService.sendNotification(). The controller is shielded from all this complexity — it calls one method and gets a Bill.

Behavioural Pattern Questions

Q10. What is the Strategy pattern? Give an example of where it applies in EnergyGrid.

A: Strategy defines a family of algorithms, encapsulates each, and makes them interchangeable. In EnergyGrid: if different payment methods (ONLINE, CASH, CHEQUE, AUTODEBIT) have different processing logic, you would create a PaymentStrategy interface with a process(Payment) method, and OnlinePaymentStrategy, CashPaymentStrategy as concrete strategies. PaymentService would select the strategy based on PaymentMethod enum. Spring uses Strategy for AuthenticationProvider (JWT vs JDBC vs LDAP auth), sort strategies in JPA queries, and serialisation strategies in Jackson.

Q11. What is the Observer pattern? How does Spring's event system implement it?

A: Observer defines a one-to-many dependency — when the subject changes, all observers are notified. Spring implements Observer via ApplicationEventPublisher and @EventListener. In EnergyGrid, after recording a payment, PaymentService publishes a PaymentReceivedEvent. NotificationService listens with @EventListener and sends a notification. BillingService also listens and may generate a receipt. Multiple observers react to one event without the publisher knowing about them. React's useState is also Observer — component (observer) re-renders when state (subject) changes.

Q12. What is the Template Method pattern? How does OncePerRequestFilter use it?

A: Template Method defines an algorithm skeleton in a base class and defers specific steps to subclasses. OncePerRequestFilter defines the template: doFilter() ensures the filter runs once and calls doFilterInternal() which is abstract. JwtFilter extends it and only implements doFilterInternal() — the 'once per request' guarantee is handled by the template in the base class. Other Spring examples: JdbcTemplate (defines the connection/cleanup template, you supply the SQL/callback), RestTemplate (defines HTTP mechanics, you supply URL).

Q13. What is the Chain of Responsibility pattern? Where is it used in EnergyGrid?

A: CoR passes a request along a chain of handlers — each handler processes or forwards. Spring Security's filter chain is the textbook example: each filter (JwtFilter, CorsFilter, ExceptionTranslationFilter, AuthorizationFilter) handles its concern and calls filterChain.doFilter() to pass to the next. The request reaches DispatcherServlet only after all security filters pass. EnergyGrid's GlobalExceptionHandler processes exception types in order — most specific exception type handled first, fallback to generic Exception.

Q14. What is the Repository pattern? How is it different from just using JPA directly?

A: Repository encapsulates data access logic — the rest of the application talks to the repository interface, not to the database directly. Without it: services would have EntityManager calls, JPQL strings, and transaction management scattered everywhere — tightly coupled to JPA. With Repository: BillingService calls billRepository.findByAccountId() — it does not know if data comes from MySQL, PostgreSQL, or an in-memory mock. Spring Data JPA's JpaRepository extends this pattern — your repository interfaces declare the methods, Spring generates the implementation.

Q15. What is the State pattern? How do Bill status and OutageEvent status demonstrate it?

A: State allows an object to change its behaviour when its internal state changes. Bill status demonstrates State: in GENERATED state, you can pay it; in PAID state, attempting payment throws BadRequestException. In DISPUTED state, billing operations are blocked. Each state has different valid transitions and different responses to the same method call. OutageEvent: in ACTIVE state, you can restore it; in RESTORED state, restoreOutage() throws BadRequestException. FieldDispatch: DISPATCHED → ONSITE → RESOLVED — only forward transitions allowed. State pattern makes these business rules explicit and maintainable.

Q16. What is the Dependency Injection pattern and how is it different from the Service Locator pattern?

A: DI is when dependencies are PUSHED into a class by an external container (Spring). The class declares what it needs (@Autowired) and Spring injects it. Service Locator is when a class PULLS its dependencies from a registry: BillingService service = ServiceLocator.get(BillingService.class). DI is preferred: classes are not coupled to the locator, dependencies are explicit (visible in constructor/fields), easy to mock in tests. Service Locator hides dependencies and is harder to test. Spring uses DI exclusively; ApplicationContext.getBean() would be Service Locator anti-pattern if called from business code.

Q17. What are the differences between Structural patterns — Adapter, Decorator, Proxy, Facade?

A: Adapter: converts an incompatible interface to a compatible one — makes things work together. Decorator: wraps an object to ADD new behaviour — same interface, enhanced functionality. Proxy: wraps an object to CONTROL access — same interface, added access control, lazy loading, or caching. Facade: provides a SIMPLIFIED interface to a complex subsystem — hides complexity. In EnergyGrid: DTO mapping = Adapter. @Transactional = Decorator. Spring AOP LoggingAspect = Proxy. BillingService = Facade. Key: Decorator and Proxy have the same interface as the wrapped object; Adapter changes the interface; Facade creates a new simplified interface.

SECTION 8 — QUICK REFERENCE CHEAT SHEET

All 23 Design Patterns — GoF Classification

Category	Pattern	Problem It Solves	Key Keyword
CREATIONAL	**Singleton**	Ensure only one instance exists	**One instance**
CREATIONAL	**Factory Method**	Defer object creation to subclass/method	**createXxx()**
CREATIONAL	**Abstract Factory**	Create families of related objects	**Factory of factories**
CREATIONAL	**Builder**	Construct complex objects step by step	**@Builder, .build()**
CREATIONAL	**Prototype**	Clone existing objects	**Cloneable, toBuilder()**
STRUCTURAL	**Adapter**	Make incompatible interfaces work	**Wrapper, DTO mapping**
STRUCTURAL	**Bridge**	Separate abstraction from implementation	**Interface + impl**
STRUCTURAL	**Composite**	Tree structure — treat part and whole uniformly	**FilterChain**
STRUCTURAL	**Decorator**	Add behaviour without subclassing	**@Transactional, @Cacheable**
STRUCTURAL	**Facade**	Simplified interface to complex subsystem	**BillingService**
STRUCTURAL	**Flyweight**	Share common state to save memory	**String pool, enum constants**
STRUCTURAL	**Proxy**	Control access to an object	**Spring AOP, Hibernate lazy proxy**
BEHAVIOURAL	**Chain of Resp.**	Pass request along chain of handlers	**Security FilterChain**
BEHAVIOURAL	**Command**	Encapsulate request as object	**@Transactional rollback**
BEHAVIOURAL	**Interpreter**	Grammar for a language	**JPQL, SpEL**
BEHAVIOURAL	**Iterator**	Sequential access to elements	**Stream API, Page**
BEHAVIOURAL	**Mediator**	Centralise complex communications	**ApplicationEventPublisher**
BEHAVIOURAL	**Memento**	Capture and restore object state	**@Transactional savepoint**

BEHAVIOURAL	**Observer**	Notify dependents on state change	**@EventListener, React useState**
BEHAVIOURAL	**State**	Change behaviour based on state	**Bill status, OutageEvent status**
BEHAVIOURAL	**Strategy**	Interchangeable algorithms	**AuthenticationProvider, Sort**
BEHAVIOURAL	**Template Method**	Algorithm skeleton with deferred steps	**OncePerRequestFilter**
BEHAVIOURAL	**Visitor**	Add operations to objects without modifying	**JPA Criteria visitor**

SOLID + Patterns Combined — Interview One-Liners

Question	One-Line Answer
Which pattern violates SRP?	God Object / Blob class — one class doing everything
Which pattern supports OCP?	Strategy, Decorator, Observer — extend without modifying
Which pattern in Spring is DIP?	IoC container + @Autowired — depend on abstractions
Singleton vs Prototype in Spring?	Singleton = one shared instance; Prototype = new per request
Decorator vs Proxy?	Decorator adds behaviour; Proxy controls access
Facade vs Adapter?	Facade simplifies; Adapter converts incompatible interface
Factory vs Builder?	Factory = one call; Builder = multi-step construction
Observer in React?	useState + useEffect — component re-renders on state change
Chain of Resp in Spring?	Security FilterChain — each filter handles then passes along
State pattern in EnergyGrid?	Bill GENERATED->PAID->DISPUTED — different behaviour per state
Template Method example?	OncePerRequestFilter — base defines skeleton, JwtFilter fills steps
Repository pattern purpose?	Decouple data access — swap MySQL for H2 in tests without code change

INTERVIEW TIPS FOR DESIGN PATTERNS & SOLID: (1) Always name the pattern AND explain the problem it solves — not just a definition. (2) Always give an EnergyGrid example — 'In my project, @Builder Lombok implements the Builder pattern on all entities.' (3) For SOLID: give a violation example first, then show how your code avoids it. (4) Spring Framework uses EVERY GoF pattern — know which Spring annotation maps to which pattern. (5) The interviewer may ask: 'Is Proxy same as Decorator?' — know the difference: Proxy controls access, Decorator adds behaviour. (6) Singleton, Factory, Builder, Decorator, Proxy, Observer, Strategy, Template Method, and Chain of Responsibility are the most commonly asked patterns in Java Spring interviews.